

OWASP Juice Shop Security Assessment Report

Author

Indra Risqiawan

July 2026

1. Executive Summary

A security assessment was conducted against OWASP Juice Shop within a personal homelab environment to gain practical experience in web application security testing. The objective of this assessment was to identify common web vulnerabilities, validate findings, and document remediation recommendations following industry best practices.

Three security findings were identified during the assessment:

- a. Authentication Bypass via SQL Injection
- b. Reflected Cross-Site-Scripting (XSS)
- c. Insecure Direct Object Reference (IDOR)

All testing activities were performed exclusively against OWASP Juice Shop, an intentionally vulnerable application designed for educational purposes

2. Objective

- a. Build a web security homelab
- b. Practices using Burp Suite Community Edition
- c. Identify vulnerabilities based on the OWASP Top 10
- d. Document the testing methodology and findings
- e. Provide remediation recommendation

Scope

Target

OWASP Juice Shop

Deployment

Docker

Host OS

Windows 10

Tools

Burp Suite Community Edition

Chromium

3. Lab Environmet

Component	Description
Host OS	Windows 10
Target	OWASP Juice Shop
Deployement	Docker
Browser	Chromium
Proxy	Burp Suite Community Edition

4. Methodology

The assessment followed these phase:

- a. Lab preparation
- b. Application exploration
- c. Security testing
- d. Finding validation
- e. Documentation
- f. Reporting

5. Findings

5.1. Authenticaion Bypass via SQL Injection

Severity: High

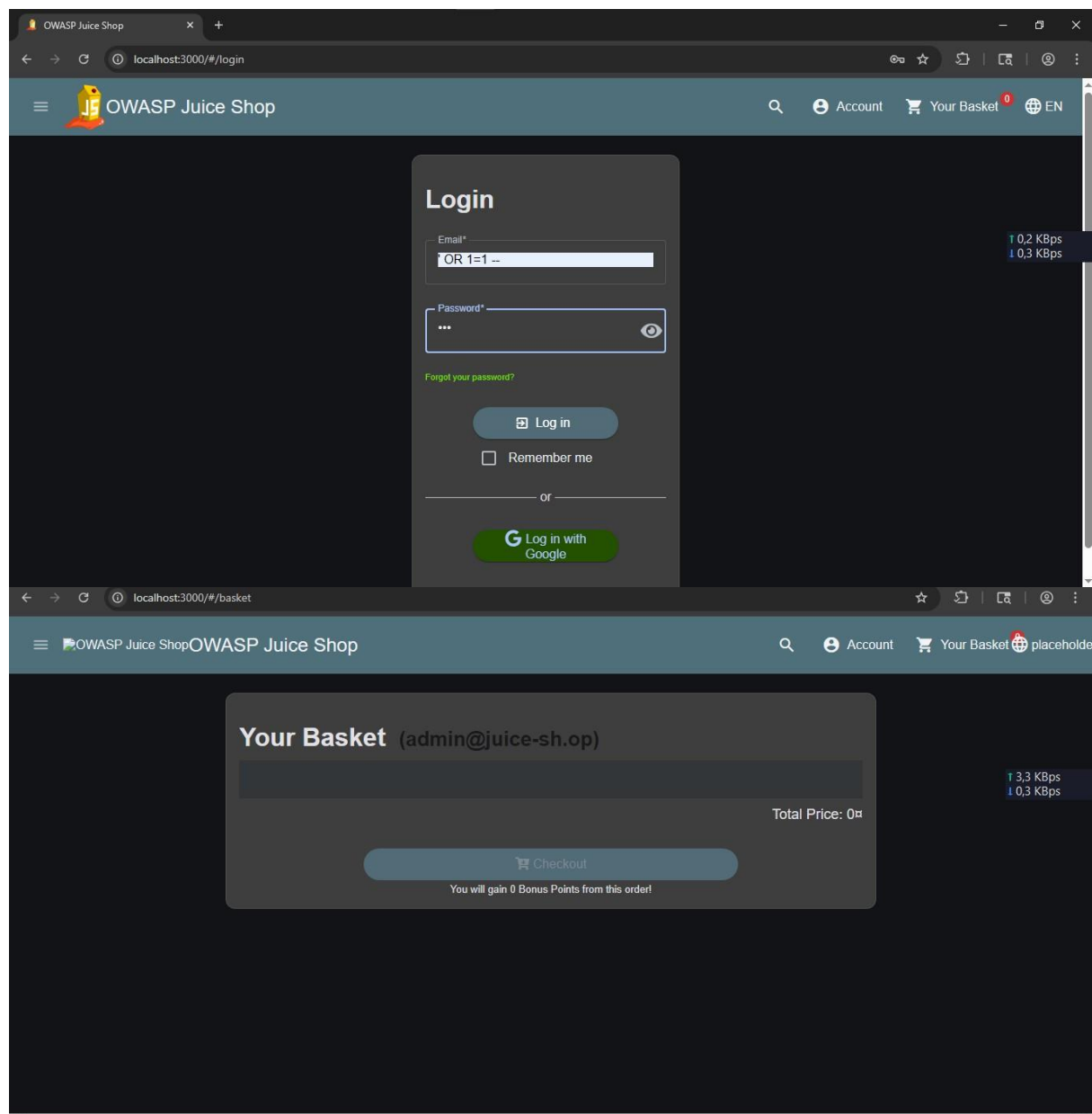
Description

A weakness in the authentication mechanism allowed administrator login to be bypassed through crafted input in the intentionally vulnerable training application

Reproduction

1. Open the OWSAP juice Shop login page
2. In the email field, enter the following SQL Injection payload:
`' OR 1=1 --`
3. Submit the login request through the application
4. Succes login as admin

Evidence



Impact

In a real world application, this type of vulnerability could allow unauthorized access to privileged accounts

Recommendation

- Use prepared statements
- Validate user input
- Avoid dynamically constructing SQL queries from user input

5.2. Reflected Cross-Site-Scripting (XSS)

Severity: Medium

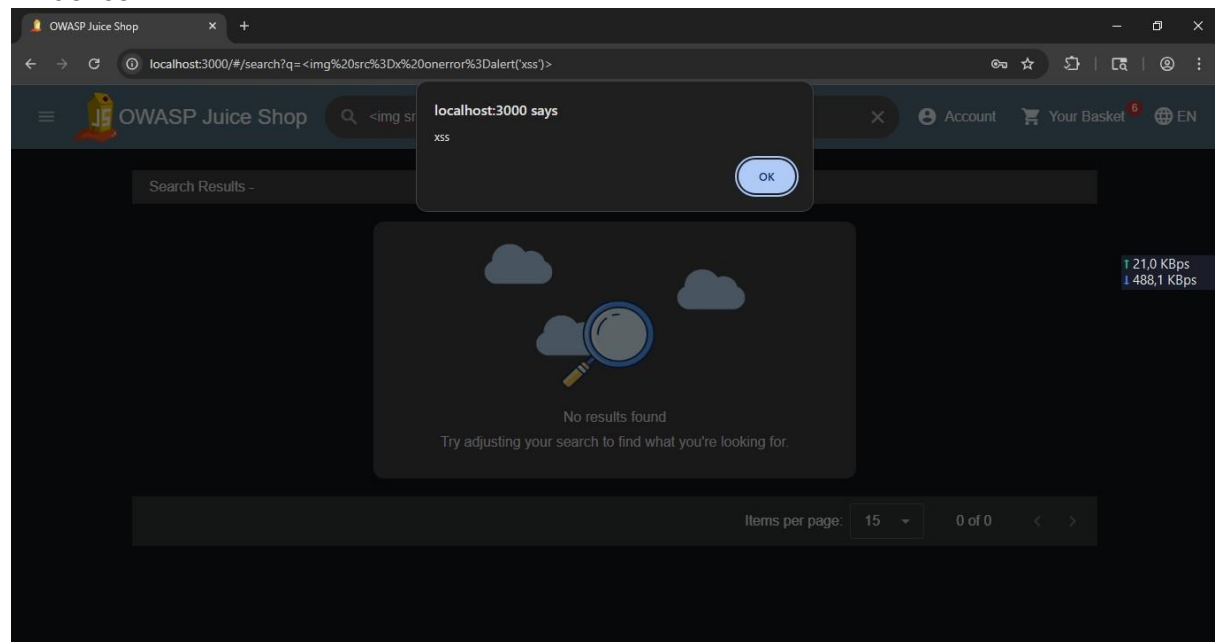
Description

User supplied input was reflected back to the browser without appropriate output encoding

Reproduction

- Navigate to the OWASP Juice Shop search feature
- Enter the following test payload into the search bar
`<img src=x onerror=alert('xss')>`
- Submit the search request

Evidence



Impact

In real application, reflected XSS may allow attacker to execute malicious JavaScript in a victims browser, potentially leading to session hijacking, phishing attack, or unauthorized actions performed on behalf of the user

- Apply output encoding
- Validate and sanitize user input
- Implement Content Security Policy (CSP)

Severity: High

The application exposed direct object references without sufficient server side authorization checks

1. Configure Burp Suite as the browser intercepting proxy
2. Enable request interception
3. Access the Basket page (<http://localhost:3000/#/basket>)
4. Intercept the HTTP request sent to the endpoint GET /rest/basket/{id}
5. Modify the {id} value to another valid basket identifier
6. Forward the modified request to the server
7. Observe that the application returns another user basket, indicating insufficient server side authorization checks

Figure1. Response for GET /rest/basket/0

The screenshot displays the Burp Suite interface during a web application security test. The main window is divided into several panes:

- Top Bar:** Shows the application name "Burp Suite Community Edition v2024.5.3 - Temporary Project" and navigation tabs for Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Discover.
- Target:** The target URL is set to "http://localhost:3000".
- Request/Response:**
 - Request:** Shows an HTTP 1.1 GET request to /test/headers. The request includes headers for Host, User-Agent, Accept, and Referer.
 - Response:** Shows an HTTP 200 OK response from localhost:3000. The response includes headers for Access-Control-Allow-Origin, X-Content-Type-Options, X-Frame-Options, Feature-Policy, X-Permitted-Cross-Domain-Policies, Content-Type, Content-Length, ETag, Vary, Date, Connection, and Keep-Alive. The response body is a JSON object: {"status": "success", "data": null}.
- Inspector:** Provides a detailed view of the request and response components, including request attributes, query parameters, body parameters, cookies, and headers.
- Status Bar:** Indicates that 416 bytes were transferred in 1,112 milliseconds.

Figure 2. Response for GET /rest/basket/1

The application returned a basket containing Apple Juice, Orange Juice, and Eggfruit Juice

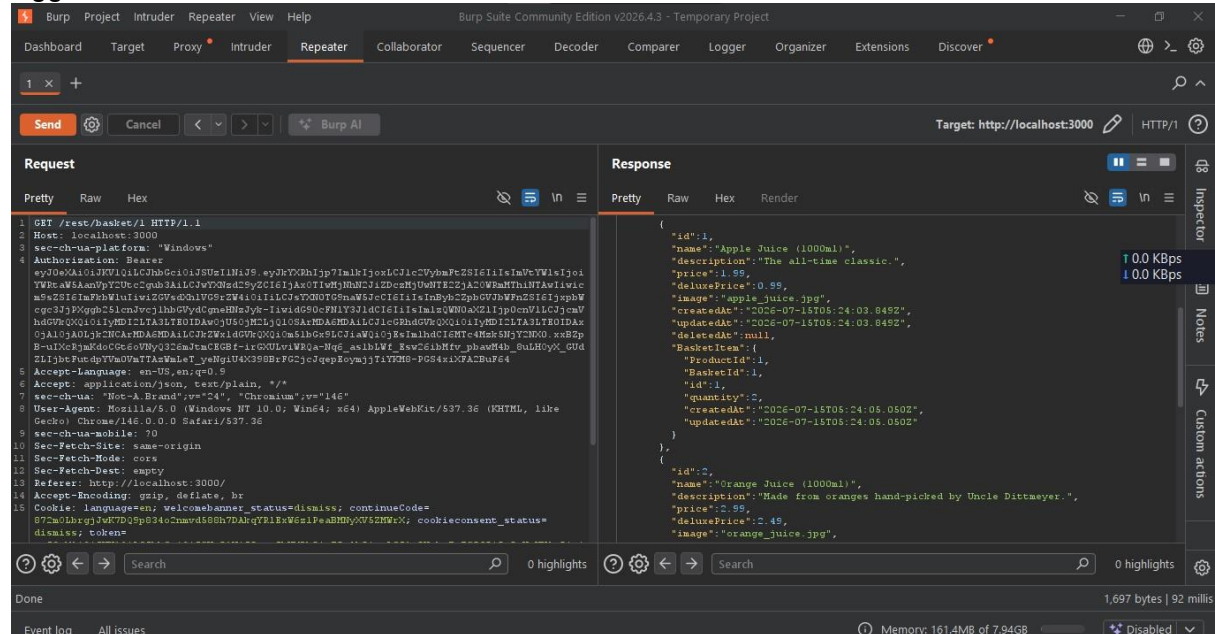
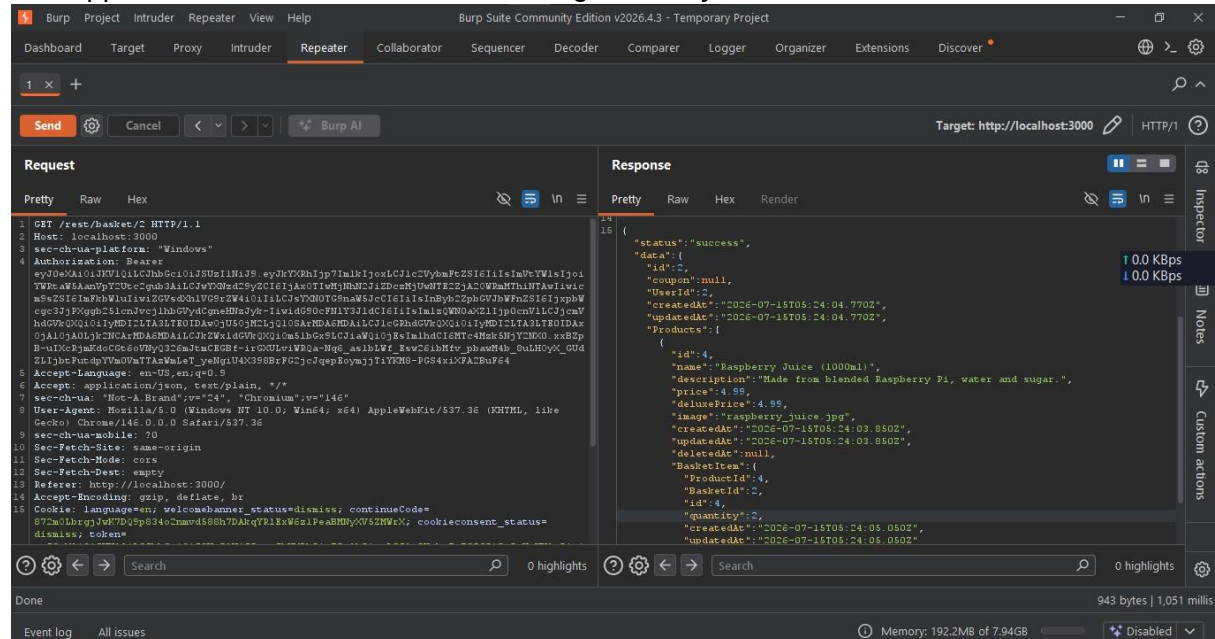


Figure 3. Response for GET /rest/basket/2

The application returned a basket containing Raspberry Juice



Demonstrating that basket contents can be accessed by by modifying the basket identifier without proper authorization

Impact

This weakness may allow unauthorized access to resources belonging to other users

Recommendation

- a. Enforces server side authorization
- b. Do not rely solely on predictable object identifiers

6. Risk Summary

Finding	Severity
Authentication Bypass via SQL Injection	High
Reflected XSS	Medium
IDOR	High

7. Conclusion

This assessment provided practical experience in identifying, validating, and documenting common web application vulnerabilities using Burp Suite within a controlled homelab environment. The project also strengthened technical reporting skills by producing structured findings and remediation recommendations. All testing activities were conducted exclusively against OWASP Juice Shop for educational purpose

8. References

- a. OWASP Top 10
- b. OWASP Web Security Testing Guide (WSTG)
- c. OWASP Juice Shop Documentation
- d. Burp Suite Documentation